

Crypto Volatility Detection in Real Time

CMU Heinz - Foundations of Operationalizing AI

Individual Programming Assignment

Author: Asli Gulcur

Date Started: November 2, 2025

Project Overview

A real-time cryptocurrency volatility detection system that:

- Ingests live crypto price data from Coinbase WebSocket API
 - Streams data through Apache Kafka
 - Engineers features to detect volatility patterns
 - Trains ML models to predict volatility
 - Tracks experiments with MLflow
 - Monitors data drift with Evidently
-

Project Structure

```
Crypto Volatility Real Time/
├── docker/                    # Docker infrastructure
│   ├── compose.yaml          # Kafka, Zookeeper, MLflow services
│   └── Dockerfile.ingestor    # Containerized ingestor
├── scripts/                  # Python scripts
│   ├── ws_ingest.py           # WebSocket data ingestor
│   ├── kafka_consume_check.py # Consumer validation
│   └── replay.py              # Feature replay for validation
├── features/                 # Feature engineering
│   └── featurizer.py          # Real-time feature calculator
├── models/                  # ML models
│   ├── train.py              # Model training script
│   ├── infer.py              # Inference script
│   ├── artifacts/            # Trained model files
│   └── data/                 # Model-specific data
├── notebooks/               # Jupyter notebooks
│   ├── eda.ipynb             # Exploratory data analysis
│   └── 03_model_training.ipynb # Model training notebook
├── docs/                    # Documentation
│   ├── feature_spec.md        # Feature specification (.md & .pdf)
│   ├── genai_appendix.md      # GenAI usage documentation
│   ├── MODEL_CARD.md          # Model documentation (.md & .pdf)
│   ├── PROJECT_EXECUTIVE_SUMMARY.md # Executive summary (.md & .pdf)
│   ├── scoping_brief.pdf      # Project scoping document
│   └── _archive/              # Archived documentation
├── reports/                 # Evaluation reports
└── evidently/               # Drift monitoring reports
```

```

├── model_evaluation_summary.md
├── data/                                # Data storage
│   ├── raw/                            # Empty (raw data stored in Kafka topic
ticks.raw)
│   ├── processed/                      # Processed feature data
├── mlruns/                             # MLflow experiment tracking
├── handoff/                             # Team project handoff package (16 files,
deployment-ready)
│   ├── compose.yaml                   # Services setup
│   ├── models/                       # Trained artifacts (2 models)
│   ├── data/                         # Sample data slices (10-min slices)
│   ├── reports/                      # Predictions & monitoring
│   ├── docs/                         # Selected documentation
│   ├── README.md                     # Handoff package documentation
│   └── generate_*.py                 # One-off scripts (used to create handoff
data)
├── config.yaml                        # ✅ Configuration file
├── .env.example                      # Environment variable template
├── requirements.txt                  # Python dependencies
├── LEARNING_TRACKER.md               # Learning journal
└── README.md                        # This file

```

📁 See `PROJECT\_STRUCTURE.md` for complete structure with file sizes and detailed descriptions.

Getting Started

Prerequisites

- Docker Desktop 4.25.2+ (for macOS 13)
- Python 3.13+
- pip 25.2+

Installation

1. Review configuration:

```

# Check default settings
cat config.yaml

# Optional: Create .env file for overrides
cp .env.example .env
# Edit .env with your custom values

```

2. Start the infrastructure:

```

cd docker
docker compose up -d

```

3. Verify services are running:

```
docker compose ps
```

You should see:

-  zookeeper (port 2181)
-  kafka (port 9092)
-  mlflow (port 5001)

4. Access MLflow UI: Open browser to: <http://localhost:5001>

Understanding Kafka Topics

Important: `ticks.raw` is a **Kafka topic**, not a file!

- **What is a topic?** A named channel inside Kafka where messages are stored
- **Where is it?** Inside Kafka's internal storage (in the Docker container)
- **How to access?** Use Kafka consumer tools (like our `kafka_consume_check.py`)
- **Not a file:** You won't see `ticks.raw` in your file system - that's normal!

Comparison:

- 📁 **FILE** (`data/raw/ticks.json`):
 - Stored on disk in file system
 - Can open with text editor
 - Simple but slow for streaming
- 📁 **KAFKA TOPIC** (`ticks.raw`):
 - Stored inside Kafka
 - Access via Kafka consumer API
 - Optimized for high-throughput streaming
 - Supports multiple concurrent readers

Python Environment

Installed packages:

- kafka-python 2.2.15
- pandas 2.3.3
- numpy 2.3.4
- scikit-learn 1.7.2
- mlflow 3.5.1
- evidently 0.7.15

Milestones

✅ Milestone 1: Streaming Setup & Scoping (COMPLETE - 11/11 tasks)

- ☒ Launch Kafka via Docker Compose
- ☒ Launch MLflow via Docker Compose
- ☒ Ingest Coinbase WebSocket data
- ☒ Implement reconnection logic
- ☒ Publish to Kafka topic `ticks.raw`
- ☒ Write consumer validation script
- ☒ Write scoping brief
- ☒ Containerize ingestor (Dockerfile.ingestor)
- ☒ Build Docker image (crypto-ingestor:latest, 145MB)
- ☒ Configure Kafka networking for Docker
- ☒ Complete 15-minute production test (2,320 messages, 0 errors)

 **Achievement:** Built production-ready real-time streaming pipeline!

✅ Milestone 2: Feature Engineering & EDA (COMPLETE - 12/12 tasks)

- ☒ Create volatility features (13 features across 3 timescales)
- ☒ Exploratory data analysis (comprehensive EDA notebook)
- ☒ Integrate Evidently monitoring (drift detection report)
- ☒ Data-driven threshold selection (95th percentile = 0.041799)
- ☒ Multi-scale feature engineering (30s/60s/120s windows)
- ☒ Feature specification documentation (17KB, publication-quality)
- ☒ Replay validation (verified reproducibility)
- ☒ Reference features generation
- ☒ Train/test split (80/20 with stratification)
- ☒ Feature engineering pipeline (build_features.py)
- ☒ Data quality validation
- ☒ Drift monitoring setup

 **Achievement:** Publication-quality feature engineering with rigorous validation!

✅ Milestone 3: Modeling & Tracking (COMPLETE - 15/15 tasks)

- ☒ Build ML models (3 models: Z-Score, Logistic Regression, Random Forest)
- ☒ Track experiments with MLflow (3+ runs logged)
- ☒ Evaluate and select production model (Random Forest: F1=0.9984, PR-AUC=1.0)
- ☒ Inference speed validation (0.004ms per sample, 4271x faster than required)
- ☒ Model card documentation (follows industry standards)
- ☒ Model artifacts saved (2 models in handoff)
- ☒ Hyperparameter tuning
- ☒ Cross-validation
- ☒ Performance benchmarking
- ☒ Test suite creation (MLflow validation, speed tests)
- ☒ Model evaluation reports (comprehensive analysis)

-  Predictions generation (32,233 predictions)
-  Model comparison analysis
-  Production deployment preparation
-  Team handoff package (16 files, 100% complete)



Achievement: Near-perfect model performance with production-ready deployment!



Configuration

Configuration Hierarchy

This project uses a **hybrid configuration approach** combining the best of both worlds:

1. **config.yaml** - Default settings and project-wide parameters
2. **Environment variables** - Runtime overrides (take precedence)
3. **.env file** - Local environment-specific settings (ignored by git)

Why this approach?

-  **Declarative defaults** in **config.yaml** for clarity and documentation
-  **Runtime flexibility** via environment variables for Docker/production
-  **Security** - secrets never committed (use .env or env vars)
-  **12-Factor App compliance** - environment-based configuration

Configuration Files

config.yaml - Contains all default settings:

- Kafka configuration (bootstrap servers, topics, timeouts)
- WebSocket settings (URL, symbols, reconnection logic)
- MLflow tracking URI and experiment names
- Feature engineering parameters (window sizes, thresholds)
- Model hyperparameters (Random Forest, Logistic Regression)
- Data paths and split ratios
- Monitoring and logging configuration

.env.example - Template for environment overrides:

```
# Copy to .env and customize
cp .env.example .env

# Example overrides:
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
MLFLOW_TRACKING_URI=http://localhost:5001
MODEL_SPIKE_THRESHOLD=0.041799
```

Best Practice:

- Use **config.yaml** for non-sensitive defaults

- Use `.env` or environment variables for secrets and deployment-specific values
- Never commit `.env` to git (already in `.gitignore`)

Example Usage

```
# In your Python scripts
import yaml
import os

# Load config.yaml
with open('config.yaml', 'r') as f:
    config = yaml.safe_load(f)

# Override with environment variables
kafka_servers = os.getenv('KAFKA_BOOTSTRAP_SERVERS',
                           config['kafka']['bootstrap_servers'])
```

Useful Commands

Docker Commands

```
# Start all services
docker compose up -d

# Stop all services
docker compose down

# View logs
docker compose logs -f kafka
docker compose logs -f mlflow

# Check status
docker compose ps
```

Kafka Commands (once scripts are ready)

```
# Run ingestor
python scripts/ws_ingest.py

# Run consumer validator
python scripts/kafka_consume_check.py
```

Learning Resources

- **LEARNING_TRACKER.md** - Detailed learning journal with concepts, progress, and notes
 - **Coinbase WebSocket API:** <https://docs.cloud.coinbase.com/exchange/docs/websocket-overview>
 - **Kafka Documentation:** <https://kafka.apache.org/documentation/>
 - **MLflow Guide:** <https://mlflow.org/docs/latest/index.html>
-



Notes

This is a learning project for the "Foundations of Operationalizing AI" course at CMU Heinz. The focus is on building production-grade streaming ML systems, not just model accuracy.

Key Learning Goals:

- Real-time data ingestion
- Stream processing with Kafka
- ML experiment tracking
- Data drift monitoring
- Production deployment patterns